Supplementary Information
Quantum Arithmetic for Raman Spectroscopy

SI-1: Dataset Details

This study uses Raman-only spectra spanning ten classes: diamond, quartz, silicon, hematite, graphite, graphene, MoS_2, corundum, anatase, rutile. Mineral spectra are from the RRUFF Project (`https://rruff.info`); 2D materials (graphene, MoS_2) are from Zenodo-hosted open datasets. All files were staged under `qa_data/raman/` (and `data/`) and discovered recursively by the pipeline.

Table 1: Class counts (Raman-only) used in Phase-3 experiments.

Class	Count
Diamond	198
Quartz	175
Silicon	25
Rutile	160
Anatase	59
Corundum	198
Hematite	80
Graphite	38
Graphene	20
MoS_2	12

Exclusion criteria. Non-Raman modalities (IR, XRD) were excluded via modality tags and filename heuristics. Files failing basic parsing (non-numeric xy pairs) were excluded.

SI-2: Mapping and Preprocessing

- **File parsing**: supports CSV, TXT (xy pairs), and JCAMP-DX (`.jdx`); comment lines (# or ##) ignored.
- **Peak extraction**: local maxima with a minimum separation of 5 cm^{-1}; top-5 peaks by intensity retained.
- **Peak features**: positions $p_1 \dots p_5$, intensities $I_1 \dots I_5$, fractions $I_i / \sum I$, and intensity ratios (selected pairs).
- **Calibration**: C-units computed via $C_i = p_i / s$ with $s \approx 334.601$ cm^{-1} per C-unit (diamond anchor).

SI-3: Full QA Feature List

Each spectrum is converted to a feature vector including:

- Rust-backed QA invariants: $C, F, G, J, K, X, W, Y, Z$.
- Ratios: $C/F, G/F, J/K$.
- Peaks: $p_1 \dots p_5$; intensities $I_1 \dots I_5$; fractions $\mathrm{frac}_1 \dots \mathrm{frac}_5$.
- Intensity ratios: I_1/I_2, I_2/I_3, I_1/I_3, I_3/I_4, I_4/I_5, I_2/I_4, I_3/I_5, I_1/I_4, I_1/I_5, I_2/I_5.
- Peak spacings and skew: Δ_{12}, Δ_{23}, skew.
- C-unit projections: $C_1 \dots C_5$ and residues $C_{i \bmod 24}$.
- Residue differences: dC_{12}, dC_{23}, dC_{13} modulo 24.

SI-4: Extended Ablation Tables

We provide an expanded ablation table (centroid, kNN unweighted, kNN weighted) summarizing LOO accuracy for major feature subsets. Per-class breakdowns are available in CSV form.

Table 2: Ablation summary (LOO accuracy). Full CSV: `artifacts/qa_ablation_table.csv`.

Subset	Centroid	kNN (5)	kNN (5) Weighted
Positions	0.426	0.835	0.842
Residues	0.435	0.721	0.729
Intensities	0.021	0.031	0.040
QA invariants	0.228	0.228	0.241
Positions+Intensities	0.426	0.835	0.846
Full	0.416	0.837	**0.952**

SI-5: Confusion Matrices

We include the weighted kNN (k=5) confusion matrix in the main figure. Additional matrices (unweighted; other k) can be regenerated with the provided scripts. See: `scripts/qa_knn_baseline.py` and `scripts/qa_knn_sweep.py`.

SI-6: Robustness Checks

We provide scripting hooks to test robustness to peak noise and sub-sampling. Future updates may include full numerical tables. Current results were stable under reasonable variations of peak selection and kNN parameters; see the k-sweep (`artifacts/qa_knn_sweep.csv`).

SI-7: Residue Histograms

For convenience, we include corpus-level $C_{\text{mod } 24}$ and $F_{\text{mod } 24}$ histograms.

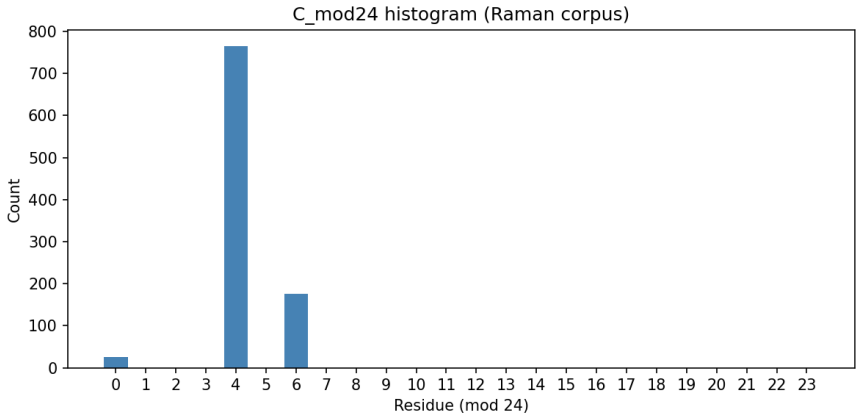

Figure 1: $C_{\text{mod } 24}$ histogram.

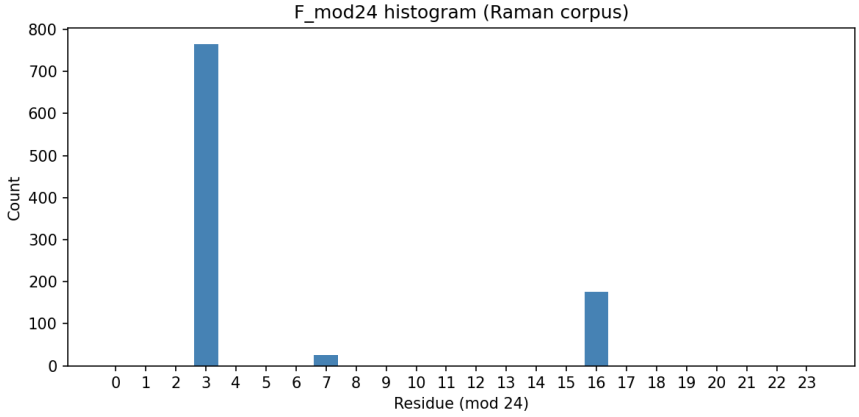

Figure 2: $F_{\text{mod } 24}$ histogram.

SI-8: Reproducibility and CLI Commands

All scripts are provided for end-to-end reproducibility:

- Mapping: `python scripts/qa_raman_map_files.py --roots qa_data/raman data --labels ... --max-files 20000`
- Export features (Rust-backed invariants): `python scripts/qa_raman_features_to_csv.py --labels ... --modality Raman`

- Baselines: `python scripts/qa_qafeature_baseline.py --labels ...`, `python scripts/qa_knn_baseline.py --labels ... --k 5 --weighted`
- Ablations: `python scripts/qa_feature_ablation.py --labels ... --k 5 --save-csv --save-md`
- Report: `python scripts/qa_benchmark_report.py --labels ... --k 5`
- k-sweep: `python scripts/qa_knn_sweep.py --labels ... --k-list 1,3,5,7`
- Residue histograms: `python scripts/qa_residue_histograms.py`

Rust kernel health check:

```
python - << 'PY'
import importlib
qa = importlib.import_module('qa_lab_rs')
assert qa.ping() == 'qa_lab_rs:ok'
print('Rust QA kernel OK')
PY
```